Inference Economics

Engineering LLMs for Cost, Latency, and Scale

Devang Sharma
Senior AI Engineer, Meta

LLM Day San Francisco 2026 | Track 2 | Harness HQ, 55 Stockton St

THE INDUSTRY

Everyone shipped AI. Then the bill arrived.

The 2026 mood, in one meme.



Inference quietly became the biggest cost line in most AI products.

Models got bigger. Traffic grew. Bills scaled linearly while margins collapsed.

The teams that win will not have the biggest models. They will be the ones serving them 10x cheaper without losing quality.

THE INDUSTRY

Where the money moved

Training was the bill. Then serving took over.

2023

The capability race

GPT-4, everyone ships a demo.

Money: training.

2024

The reasoning turn

Models think before answering.

Context windows explode.

Money: still training.

2025

Agents reach production

One user request becomes

a dozen model calls.

2026

Serving dominates

Reasoning x agents x context.

The budget moved to inference.

Training is capex: you pay once. Inference is opex: it arrives on every request, every day, forever.

Three multipliers stacked on the same bill: reasoning emits many more tokens per question, agents multiply calls per request, and context windows went from 4K to a million. None were mistakes. All were capability wins that cost tokens.

FOUNDATIONS

What is inference, really?

The simplest definition you will hear today.

Inference is running a trained model to answer a question.

No training. No learning. Just answering, live, over and over, as fast as possible.

You ask
a question,
a prompt

→

The model thinks
billions of tiny
math operations

→

Answer streams back
one word
at a time

Every chatbot reply, every agent step, every autocomplete: that is inference. And you pay for each one.

FOUNDATIONS

Two halves, two speed limits

This is the one mental model the whole talk depends on.

It has two halves:

reading your prompt (fast, math-heavy)

and writing the answer one word at a time (slow, because the GPU re-reads the whole model from memory for every single word).

1 Reading your prompt



whole prompt, all at once | math-heavy, fast

2 Writing the answer



one word at a time | re-reads the model per word

“If the GPU has to read the entire model to write one word for one user, it may as well write a word for a hundred users in that same read.”

WHY IT MATTERS

Why should you care?

Because this is where your margin goes.

$57,600 a month

to answer 35 questions a second. One model. Not a hyperscaler, a mid-size product.

- You pay per answer, not per user. More users means thinner margins.
- Speed and cost fight each other. Every saving trades against waiting time.
- The out-of-the-box setup is the expensive one. Nobody ships a tuned fleet by accident.



The winners will serve AI ten times cheaper at the same quality. That is engineering, not luck.

AGENDA

The plan

Problem, approach, live demo, then the playbook.

1

The problem

The exact question, stated precisely

2

The approach

A flight simulator for your GPU bill

3

Live demo

A real run, real metrics, no code tour

4

Results + playbook

What moved, what failed, what to steal

5

Q&A

Bring your worst inference bill

THE PROBLEM

The question nobody can answer

“How many tokens per second can we do?”

That number is cherry-pickable. Quietly let replies get slower, and it improves. Teams end up arguing about benchmarks that were never comparable.

The question that has an answer:

How many GPUs, to hold this latency, at this traffic?

Fix the traffic. Fix the latency promise. Then the only thing left to vary is the engineering.

Latency is an output, never an input. Queueing and contention produce it. Assume it, and the model can only tell you what you already believed.

THE APPROACH

A flight simulator for your GPU bill

Capacity planning before you rent a single GPU.

1

Give it your traffic shape and your latency SLO.

2

It searches replica counts
until both targets hold.

3

Flip a lever, re-run,
read the new GPU count.

Why trust it

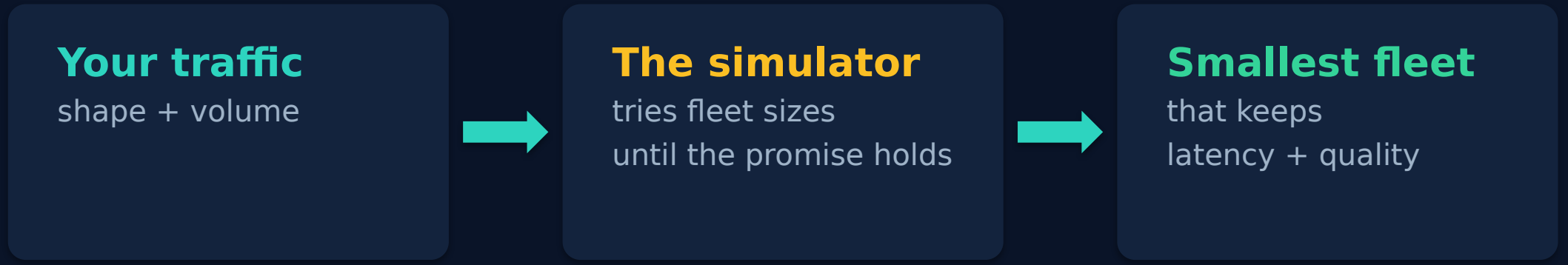
- Mechanisms are actually implemented, not fudge factors.
- Hardware math from published specs.
- Checked against 11 published reference points: 11 of 11 reproduced, worst error 6.5%.

Answers it on a laptop in about 20 seconds. No GPU, no network, no API key.

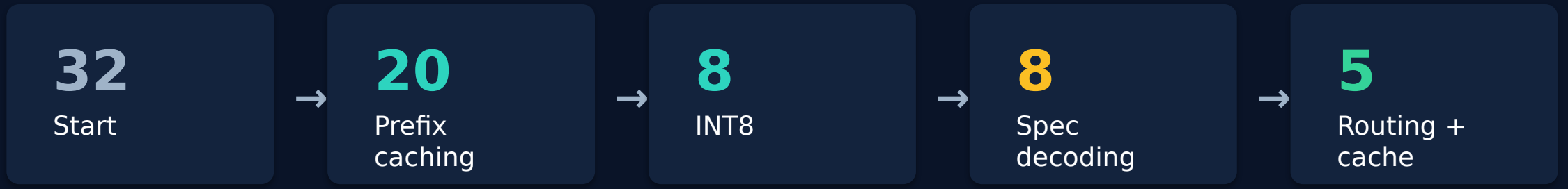
THE APPROACH

What we built

A capacity-planning tool, in one picture.



Then we flip the five levers on, one by one, and watch the GPU count fall:



Same traffic, same promise. The only thing that changes is the engineering.

THE APPROACH

Five levers, five different bottlenecks

Which is exactly why the order matters.

Prefix caching
Stop recomputing the same system prompt and retrieved context on every request.

Quantization (INT8)
Make the model physically smaller in memory, so every read is cheaper.

Speculative decoding
A small model guesses ahead; the big one checks several words at once.

Routing + response cache
Easy questions go to a smaller model. Repeats skip the GPU entirely.

Survival
What happens when traffic doubles with no warning.

Watch the GPU column in the demo. One of these five is going to disappoint us.
That will be the most useful slide in the deck.

LIVE DEMO

What we are about to run

Five configurations, one traffic shape, one promise that never moves.

The traffic

3,000 requests over 86 seconds = 34.8 req/s

40% chat | 30% RAG | 18% agent | 12% code

The promise (never moves)

p95 first token ≤ 1 second

per-token ≤ 50 ms

The fleet search must hold both, every run.

The hardware

H100 SXM, 3.35 TB/s memory bandwidth

Llama-70B | $2.50 per GPU-hour

KV cache: 320 KiB per token

Runs: stock vLLM → + prefix cache → + INT8 → + speculative decoding → + routing & cache → spike test (2x traffic)

LIVE DEMO

```
python demo.py
```

github.com/Devang-25/inference-economics

Five minutes. I will not tour the code. I will run it, show the metrics, and the repo is yours afterwards.

RESULTS

32 GPUs → 5. $57,600 → $9,000 a month.

Same traffic. Same latency promise, kept at every step. Quality cost: 0.31%.

Stage	GPUs	$ / month	Key number
Stock vLLM defaults	32	$57,600	39 requests in flight
+ Prefix caching	20	$36,000	64% of prompt tokens not recomputed
+ INT8 quantization	8	$14,400	98 requests in flight
+ Speculative decoding	8	$14,400	63% accepted, no gain
+ Routing + response cache	5	$9,000	68% to small model, 26% skip GPU

Final: p95 first token 60 ms, per-token 17.8 ms. $48,600 a month saved.

RESULTS

What surprised us

The two slides most teams get wrong.

The lever that failed

Speculative decoding: 8 GPUs → 8. Per-token latency slightly worse.

It worked as designed: 63% of guesses accepted.

Why: the trick is free only while memory-bound. At 98 requests in flight we are compute-bound, and the extra guesses compete with real work.

A latency tool for low batch. First thing to switch off for cheap volume.

The spike test: traffic doubles

Serve everyone: first token 34.9 s. Nobody turned away, nobody served. A capacity problem becomes a total outage.

Protect the promise: cap concurrency, turn away at the door. 0.9 s, 645 served properly, the rest get a fast honest no.

Autoscaling cannot save you: loading a 70B onto cold GPUs takes minutes. The spike is now.

WRAP UP

The 10x Inference Playbook

Steal this playbook.

0 Measure

Cost per million tokens, per route. You cannot manage what you never measured.

1 Find your ratio

Prompt tokens vs answer tokens. That one number tells you which bottleneck you have.

2 Turn on what is free

Caching and batching. No quality cost.

3 Shrink the model

Quantization. The biggest single technical win.

4 Route

The largest remaining lever is architectural, not kernel-level.

5 Clever tricks last

Measured on your own batch sizes. See slide 15.

Where this goes next: capacity planning before you buy GPUs | a cost gate in CI | build-vs-buy with your own numbers | MoE | disaggregated prefill and decode

Questions?

Bring your worst inference bill.

`github.com/Devang-25/inference-economics`

Devang Sharma | Senior AI Engineer, Meta
linkedin.com/in/idevangsharma